Độc lập – Tự do – Hạnh phúc

ĐỀ KIỂM TRA KẾT THÚC HỌC PHẦN

Học kỳ: 1 Năm học: 2025 – 2026

Học phần: Thiết kế, phát triển và bảo trì phần mềm (SEE522)

Thời gian làm bài: 90 phút (không kể thời gian phát đề)

(Sinh viên được sử dụng tài liệu giấy)

Họ, tên sinh viên: Hồ Thanh Hân **Mã số:** DTM225609

Câu 1 (1 điểm): Hãy sử dụng phương pháp inline method để chỉnh sửa cho đoạn mã bên dưới trở nên gọn gàng và dễ bảo trì hơn.

```
class PizzaDelivery {
    int getRating() {
        return moreThanFiveLateDeliveries() ? 2 : 1;
    }
    boolean moreThanFiveLateDeliveries() {
        return numberOfLateDeliveries > 5;
    }
}
```

Câu 2 (2 điểm): Chúng ta đang đối mặt với bài toán tính lương nhân viên trong một công ty có cấu trúc phòng ban và nhân viên phức tạp. Để giải quyết vấn đề này, chúng ta quyết định sử dụng Composite Pattern, một mẫu thiết kế giúp tạo cấu trúc cây của các đối tượng để chúng có thể được xử lý một cách thống nhất.

Mỗi nhân viên trong công ty được biểu diễn thông qua đối tượng "Employee", chứa thông tin như tên và số tiền lương. Các phòng ban, hay còn được biểu diễn bằng đối tượng "Department", có khả năng chứa cả nhân viên lẫn các phòng ban khác.

Gợi ý sử dụng mẫu thiết kế Composite: Interface IEmployee định nghĩa chung cho các thuộc tính "Name" kiểu string và hàm "CalculateSalary()" trả về kiểu decimal. Employee có thuộc tính "Name" và "Salary" và lớp này là nút lá (Leaf) của cây. Department là thành phần Composite có thể chứa cả Employee lẫn các Department khác. Phương thức CalculateSalary của Department sẽ tổng hợp lương của tất cả các nhân viên và phòng ban trực thuộc.

Dựa trên gợi ý, bạn hãy thiết kế một sơ đồ lớp UML sử dụng Composite Pattern để hiệu quả hóa việc tính toán tổng lương cho toàn bộ công ty, bao gồm cả nhân viên và các phòng ban. Điều

này giúp chúng ta quản lý công việc này một cách linh hoạt và dễ dàng mở rộng khi có thêm nhân viên hoặc phòng ban mới.

Câu 3 (1 điểm): Dưới đây là mô tả vắn tắt tình trạng hiện tại và yêu cầu thay đổi. Hãy phân loại yêu cầu thay đổi này.

- Hiện tại: Phần mềm lưu số chi nhánh bởi mã số có 2 chữ số.
- Yêu cầu: Sau này, công ty sẽ mở rộng thì số chi nhánh lên hàng trăm, phần mềm phải có khả năng thích nghi với số lượng chi nhánh mới.

Câu 4 (3 điểm): Cho đoạn chương trình C

```
//------
Void SapXep (int arr[], int n)
{
        int tam;
        if (n > 0)
        {
                for (int i=0; i<n; i++)
                        for (int j=0; j<i; j++)
                                if(arr[i] <arr[j])
                                {
                                        tam=arr[i];
                                        arr[i]= arr[j];
                                        arr[j]= tam;
                                 }
        }
        else  exit();
}
//--------
```

a. Hãy xác định độ phức tạp của chương trình theo phương pháp McCabe với 3 cách thức (công thức, số vùng liên thông và số điều kiện).

b. Hãy xác định độ phức tạp của chương trình theo Halstead.

Câu 5 (3 điểm): Nhóm của bạn được giao bảo trì một sản phẩm phần mềm. Sau khi phân tích, nhóm quyết định sẽ:

- **Sửa chữa một số chức năng** với 13 EI đơn giản, 2 EI phức tạp, 2 EO trung bình, 3 EO phức tạp, 2 EQ phức tạp, 3 ILF đơn giản, 2 ILF trung bình, và 3 EIF phức tạp.

a. Hãy tính số lượng điểm chức năng thô.

b. Nếu hầu hết các hệ số kỹ thuật đều không quan trọng. Ngoại trừ hệ số dễ vận hành là cực kỳ quan trọng. Hãy tính số lượng điểm chức năng được điều chỉnh là bao nhiêu?

c. Hãy qui đổi ra số lượng dòng mã lệnh theo ngôn ngữ lập trình C#.

d. Giả sử chương trình có 6.35 ngàn dòng lệnh (kích thước mã lệnh cơ sở). Sau khi phân tích thì phần mềm có cấu trúc ở mức cao, còn lập trình viên thì một chút biết rõ. Giả sử các thông số nhìn chung đều ảnh hưởng ở mức bình thường nhưng sản phẩm có tính chất phức tạp vô cùng cao. Sử dụng mô hình COCOMO II để xác định nỗ lực bảo trì và tính tổng chi phí bảo trì nếu 1 tháng phải trả 5 triệu đồng cho 1 người.

---------- HẾT ----------

Câu 1:

```
class PizzaDelivery {
    int getRating() {
    return (numberOfLateDeliveries > 5)
                ? 2 : 1;
}}
```

Câu 4:

a) McCabe Complexity

V(G) = E - N + 2 = số đường viền + 1

→ có 1 if, 2 vòng for, 1 else

→ 4 nhánh → V(G) = 5

b) Halstead:

- $n_1 = 7$
- $n_2 = 6$
- $N_1 = 13, N_2 = 10$ (với lượng)

→ các giá trị khác (V, D, E) có công thức sau:

- $V = (N_1 + N_2) * \log_2(n_1 + n_2)$
- $D = (n_1/2) * (N_2/n_2)$
- $E = V * D$

=> V = thay thử các số thì được: $V = 85,11$, $D = \frac{35}{6}$, $E = 496,475$

Câu 5: